

Factory Pattern in JavaScript

What is the Factory Pattern?

The **Factory Pattern** is a creational design pattern that provides an interface for creating objects without exposing the exact object creation logic to the code that uses them.

In simple terms:

"Ask the factory for an object, and it decides what type of object to create."

Instead of directly creating objects:

```
const car = new Car();  
const bike = new Bike();  
const truck = new Truck();
```

The Factory Pattern allows:

```
const vehicle = VehicleFactory.create("car");
```

The factory decides:

- Which class to instantiate.
- How the object should be configured.
- Which implementation should be returned.

Why Does the Factory Pattern Exist?

A common problem in programming is that your code becomes tightly coupled to specific classes.

Example:

```
class PaymentProcessor {  
  
  process(type){  
  
    if(type === "credit"){  
  
      return new CreditCardPayment();  
  
    }  
  
  }  
  
}
```

```
    if(type === "paypal"){  
        return new PayPalPayment();  
    }  
  
    if(type === "crypto"){  
        return new CryptoPayment();  
    }  
}  
  
}
```

Problems:

- The main code knows about every class.
- Adding new types requires changing existing code.
- Object creation logic is spread throughout the application.

The Factory Pattern moves this responsibility into one place.

Basic Factory Pattern Example

Vehicle Factory Example

Imagine an application that supports different vehicles:

- Cars
- Motorcycles
- Trucks

Without a Factory:

```
const car = new Car();  
const motorcycle = new Motorcycle();  
const truck = new Truck();
```

The application needs to know every class.

With a Factory:

```
const vehicle = VehicleFactory.create("car");
```

The factory handles creation.

Vehicle Classes

These are the objects that the factory creates.

```
class Car {  
  
  constructor(){  
  
    this.type = "Car";  
  
  }  
  
  drive(){  
  
    console.log("Driving a car 🚗");  
  
  }  
}  
  
class Motorcycle {  
  
  constructor(){  
  
    this.type = "Motorcycle";  
  
  }  
  
  drive(){  
  
    console.log("Riding a motorcycle 🏍️");  
  
  }  
}  
  
class Truck {  
  
  constructor(){
```

```
    this.type = "Truck";  
  
  }  
  
  drive(){  
  
    console.log("Driving a truck 🚚");  
  
  }  
  
}
```

Factory Class

The Factory decides which object to create.

```
class VehicleFactory {  
  
  static createVehicle(type){  
  
    switch(type){  
  
      case "car":  
  
        return new Car();  
  
      case "motorcycle":  
  
        return new Motorcycle();  
  
      case "truck":  
  
        return new Truck();  
  
      default:  
  
        throw new Error("Unknown vehicle type");  
  
    }  
  
  }  
  
}
```

```
    }  
  }  
}
```

Using the Factory

```
const vehicle1 = VehicleFactory.createVehicle("car");  
  
const vehicle2 = VehicleFactory.createVehicle("truck");  
  
vehicle1.drive();  
  
vehicle2.drive();
```

Output:

```
Driving a car 🚗  
  
Driving a truck 🚚
```

The application does not need to know:

```
new Car()  
new Truck()
```

The factory handles that.

How This Works Step-by-Step

1. Request an Object

The application calls:

```
VehicleFactory.createVehicle("car");
```

The factory receives:

```
type = "car"
```

2. Factory Checks the Requested Type

The factory runs:

```
switch(type)
```

It checks:

```
case "car":
```

3. Factory Creates the Object

The factory creates:

```
return new Car();
```

The caller receives:

```
Car {  
  type: "Car"  
}
```

4. Application Uses the Object

The application only knows:

```
vehicle.drive();
```

It does not care whether it is:

- A Car.
 - A Motorcycle.
 - A Truck.
-

Factory Pattern Using a Function (Common JavaScript Approach)

JavaScript often uses simple factory functions instead of classes.

Example:

```
function createUser(type){  
  
  if(type === "admin"){  
  
    return {  
      role: "Admin",  
      permissions: [  
        "read",  
        "write",  
        "delete"  
      ]  
    };  
  
  }  
  
  if(type === "guest"){  
  
    return {  
      role: "Guest",  
      permissions: [  
        "read"  
      ]  
    };  
  
  }  
  
}
```

Using it:

```
const admin = createUser("admin");  
  
const guest = createUser("guest");  
  
console.log(admin.permissions);  
  
console.log(guest.permissions);
```

Output:

```
[
  "read",
  "write",
  "delete"
]

[
  "read"
]
```

A More Realistic JavaScript Example

Notification Factory

A real application might support:

- Email notifications
- SMS notifications
- Push notifications

Instead of:

```
new EmailNotification();

new SMSNotification();

new PushNotification();
```

Use:

```
NotificationFactory.create("email");
```

Notification Classes

```
class EmailNotification {

  send(message){

    console.log(`Email sent: ${message}`);
```



```
    }  
}  
  
class SMSNotification {  
  
    send(message){  
  
        console.log(`SMS sent: ${message}`);  
  
    }  
}  
  
class PushNotification {  
  
    send(message){  
  
        console.log(`Push notification sent: ${message}`);  
  
    }  
}
```

Factory

```
class NotificationFactory {  
  
    static create(type){  
  
        switch(type){  
  
            case "email":  
  
                return new EmailNotification();  
  
        }  
    }  
}
```

```
    case "sms":  
        return new SMSNotification();  
  
    case "push":  
        return new PushNotification();  
  
    default:  
        throw new Error("Invalid notification type");  
    }  
}  
}
```

Using the Factory

```
const notification =  
    NotificationFactory.create("email");  
  
notification.send("Your order shipped!");
```

Output:

```
Email sent: Your order shipped!
```

Common Uses of Factory Pattern in JavaScript

1. Creating Different API Clients

Example:

```
ApiFactory.create("github");
```

Possible objects:

- GitHub API client
- Stripe API client
- Google API client

Problem solved:

Hide API client creation logic from the rest of the application.

2. Database Connections

Example:

```
DatabaseFactory.create("postgres");
```

Creates:

- PostgreSQL connection
- MySQL connection
- MongoDB connection

Problem solved:

Allow switching databases without changing application code.

3. User Roles

Example:

```
UserFactory.create("admin");
```

Creates:

- Admin user
- Customer user
- Moderator user

Problem solved:

Create different objects with different behaviours.

4. Game Development

Example:

```
EnemyFactory.create("boss");
```

Creates:

- Goblin
- Dragon
- Boss enemy

Problem solved:

Create many variations of objects dynamically.

5. UI Components

Example:

```
ComponentFactory.create("button");
```

Creates:

- Button
- Checkbox
- Dropdown

Problem solved:

Centralise component creation.

Problems Factory Pattern Solves

Problem 1: Tight Coupling

Without Factory:

```
class Order {  
  
    createPayment(){  
  
        return new PayPalPayment();  
    }  
}
```

```
}  
  
}
```

The Order class is permanently tied to PayPal.

With Factory:

```
class Order {  
  
    createPayment(){  
  
        return PaymentFactory.create("paypal");  
  
    }  
  
}
```

The creation logic is separated.

Problem 2: Complex Object Creation

Some objects require setup:

Example:

```
new Database(  
    username,  
    password,  
    host,  
    port,  
    settings  
);
```

Factory:

```
DatabaseFactory.create("production");
```

The setup is hidden.

Problem 3: Adding New Object Types

Without Factory:

```
if(type === "paypal")
if(type === "stripe")
if(type === "crypto")
```

Logic spreads throughout the application.

With Factory:

```
PaymentFactory.create("stripe");
```

Only the factory changes.

Factory vs Normal Class

Normal Class	Factory
Creates itself directly	Creates objects for you
Caller knows the class	Caller only knows the factory
Simple object creation	Complex/dynamic creation
More tightly coupled	Looser coupling

Factory vs Builder Pattern

Factory	Builder
Creates different types of objects	Creates one complex object
Focuses on "which object?"	Focuses on "how to build object?"
Usually one method call	Multiple construction steps
Hides object selection	Controls object construction

Example Factory:

```
VehicleFactory.create("car");
```

Question answered:

"Which vehicle should I create?"

Example Builder:

```
new CarBuilder()  
.setEngine("V8")  
.setColour("Red")  
.build();
```

Question answered:

"How should I construct this car?"

Factory Pattern Types

1. Simple Factory

The most common JavaScript version.

Example:

```
Factory.create("type");
```

A single function decides what to create.

2. Factory Method

A class defines a method for creating objects.

Example:

```
class Logistics {  
  
  createTransport(){  
  
  }  
  
}
```

Subclasses decide what object is created.

3. Abstract Factory

A factory that creates families of related objects.

Example:

```
WindowsFactory.createButton();  
  
MacFactory.createButton();
```

Used for:

- Cross-platform applications.
- UI frameworks.

Disadvantages of Factory Pattern

1. More Code

For simple objects:

```
new User();
```

is often better than:

```
UserFactory.create();
```

2. Factory Can Become Large

A factory with many object types can become:

```
switch(type){  
  
  case "A":  
  
  case "B":  
  
  case "C":  
  
  case "D":
```



```
}
```

This may need splitting into multiple factories.

3. Can Hide Simple Logic

Sometimes developers introduce factories unnecessarily.

When Should You Use Factory?

Use Factory when:

- ✓ You create different types of objects
- ✓ Object creation logic is complex
- ✓ You want to reduce coupling
- ✓ New object types may be added later

Examples:

- Payment systems
- Database clients
- API clients
- Game objects
- UI components

Avoid Factory when:

- ✗ You only create one simple object type
- ✗ The constructor is already easy to understand
- ✗ The factory adds unnecessary abstraction

Key Takeaway

The Factory Pattern separates:

```
Object Creation
  |
  v
Factory
  |
  v
Correct Object
```

Instead of:

```
Application Code
    |
    v
new SpecificClass()
```

In JavaScript, the Factory Pattern is commonly implemented using:

1. Factory classes with static methods.
2. Factory functions.
3. Switch statements or object maps to select implementations.

It is mainly used when an application needs to create **different types of objects without tightly coupling the code to specific classes**.